UNIVERSITÉ JOSEPH KI-ZERBO

(UJKZ)

******

INSTITUT BURKINABÈ DES ARTS ET MÉTIERS

(IBAM)

Master1/Filière : Sécurité des systèmes d'Informations (SI)

Cours de développement de composants web

RAPPORT DE PROJET PRATIQUE (Exercice 23):

Développement d'une API Gateway d'agrégation pour un système de santé

Réalisé par le groupe 4 *:*

DIANDE ABDOU NASSIRO

OUEDRAOGO WEND-KOUNI BASILE

OUEDRAOGO SANATA

TIONON FILWENDE MICHAEL

WANDAOGO SALIF

Professeur *:*

KY FADEL OUMAR

Année universitaire : 2025–2026

Sommaire

1. Introduction 7

2. Technologies utilisées 8

3. Architecture 9

4. Conception et réalisation 10

5. Stratégie de gestion des erreurs partielles 13

6. Délais d'attente, corrélation et journaux structurés 14

7. Déploiement 16

8. Tests et résultats 18

9. Guide de démonstration — préparer l'environnement 19

10. Guide de démonstration — les 7 scénarios 22

11. Conclusion et perspectives 27

Annexe A — Arborescence du projet VII

Annexe B — Sorties obtenues lors de la démonstration VII

Résumé

Ce rapport présente la réalisation de l'exercice 23 du projet de développement à base de composants et services web. L'objectif est de concevoir une API Gateway d'agrégation pour un tableau de bord de santé, permettant à un client d'obtenir en un seul appel les informations d'une structure sanitaire, ses indicateurs d'activité et le résumé de ses patients. Trois API indépendantes ont été développées avec FastAPI : structures sanitaires, patients fictifs et indicateurs agrégés. Un composant d'agrégation en Python, utilisant httpx en mode asynchrone, interroge ces trois services en parallèle et compose une réponse unique exposée via Kong 3.9 en mode DB-less sur l'endpoint GET /api/v1/dashboard/structures/{id}. L'ensemble est déployé dans cinq conteneurs Docker, orchestrés par Docker Compose dans une machine virtuelle Ubuntu 22.04 créée avec Vagrant.

Le projet met particulièrement l'accent sur la gestion des erreurs partielles : chaque section de la réponse porte le statut DISPONIBLE, ABSENT ou INDISPONIBLE, afin de distinguer une donnée réellement nulle d'une donnée manquante ou inaccessible. Des délais d'attente emboîtés de 2 s, 3 s et 5 s, un identifiant de corrélation X-Correlation-ID et des journaux JSON structurés assurent la prévisibilité et la traçabilité du système. La séparation des responsabilités est maintenue : Kong gère le transport, l'agrégateur orchestre la composition, et chaque API conserve sa logique métier. Les résultats obtenus confirment le bon fonctionnement du système : les 23 tests unitaires et les 14 vérifications de bout en bout exécutés lors de la démonstration ont tous réussi, couvrant les réponses complètes, les données nulles, les erreurs partielles, les délais dépassés et les indisponibilités de services. La suite compte désormais 28 tests unitaires, dont 5 sur les contrats d'API. Le projet est livré dans un dépôt Git public, avec ses bases de test, ses contrats OpenAPI, un workflow d'intégration continue et une procédure d'installation reproductible.

Liste des sigles et abréviations

Sigle / Abréviation	Signification	Précision / usage dans le rapport
API	Application Programming Interface — Interface de programmation applicative	Désigne les services exposés et consommés
API Gateway	Passerelle API	Point d'entrée unique vers plusieurs services
API REST	Application Programming Interface Representational State Transfer	Style d'API utilisé pour les trois services
CHU	Centre Hospitalier Universitaire	Type de structure sanitaire
CHR	Centre Hospitalier Régional	Type de structure sanitaire
CMA	Centre Médical avec Antenne chirurgicale	Type de structure sanitaire
CSPS	Centre de Santé et de Promotion Sociale	Type de structure sanitaire
CI	Continuous Integration — Intégration continue	Workflow GitHub Actions qui rejoue les tests à chaque push
DB	Database — Base de données	Utilisé dans « DB-less »
DB-less	Sans base de données	Mode de fonctionnement de Kong sans PostgreSQL
ELK	Elasticsearch, Logstash, Kibana	Outils de centralisation et d'analyse de journaux
HTTP	HyperText Transfer Protocol	Protocole de communication des API REST
HTTPS	HyperText Transfer Protocol Secure	Version sécurisée de HTTP
ID	Identifier — Identifiant	Utilisé dans X-Correlation-ID, structure_id, etc.
INFO	Niveau de journalisation « information »	Niveau des journaux JSON
JSON	JavaScript Object Notation	Format des données échangées et des journaux
JWT	JSON Web Token	Mécanisme d'authentification cité en perspective

OpenAPI	Spécification OpenAPI (anciennement Swagger)	Format des contrats d'API publiés dans contracts/
ms	Milliseconde	Unité des durées de réponse
REST	Representational State Transfer	Style architectural des API
TP	Travaux Pratiques	Contexte du projet
UUID	Universally Unique Identifier	Format de l'identifiant de corrélation
VM	Virtual Machine — Machine virtuelle	Environnement Ubuntu créé par Vagrant
WARNING	Niveau de journalisation « avertissement »	Utilisé lorsqu'un service est indisponible
YAML	YAML Ain't Markup Language	Format du fichier de configuration kong.yml

Liste des figures et tableaux

Liste des figures

Figure 1:Architecture du système · Kong, agrégateur, 3 API 9

Liste des tableaux

Tableau 1: TableauCorrespondance entre les exigences de l'énoncé et les réalisations du projet . 7
Tableau 2: Technologies utilisées et leur rôle dans le projet 8
Tableau 3: Endpoints et jeux de données des trois API indépendantes 10
Tableau 4: Routage et délais configurés dans Kong 12
Tableau 5: Plugins Kong : portée et effets 12
Tableau 6: Les trois statuts de réponse et leur signification 13
Tableau 7: Codes d'erreur d'indisponibilité et modes de détection par l'agrégateur 13
Tableau 8: Règles de décision : situation des sources, code HTTP et statut d'agrégation 14
Tableau 9: Délais d'attente emboîtés et comportement en cas de dépassement 14
Tableau 10: Types d'événements journalisés et informations associées 16
Tableau 11: Conteneurs Docker, images et ports exposés sur la machine virtuelle 16
Tableau 12: Couverture des exigences par les tests unitaires et les vérifications de bout en bout 19
Tableau 13: Commandes de démarrage et de connexion à la machine virtuelle 20
Tableau 14: Commandes de démarrage et de vérification de la pile Docker 20
Tableau 15: Rôle des éléments de la commande de suivi des journaux 21
Tableau 16: Outils et options utilisés dans les commandes de démonstration 22
Tableau 17: Éléments de la commande de vérification des services déclarés dans Kong 23
Tableau 18: Éléments de la commande d'appel au tableau de bord via Kong 23
Tableau 19: Éléments des commandes d'extraction des données du scénario 3 24
Tableau 20: Rôle des commandes utilisées dans le scénario 4 24
Tableau 21: Rôle des commandes utilisées dans le scénario 5 25
Tableau 22: Rôle des commandes utilisées dans le scénario 6 26
Tableau 23: Rôle des commandes utilisées dans le scénario 7 26
Tableau 24: Guide de dépannage pendant la démonstration 27
Tableau 25: Journaux de l'agrégateur pour le scénario 2 (réponse complète) IX

Tableau 26: Journaux de l'agrégateur pour la structure 5 (scénario 3)..... X
Tableau 27: Journaux de l'agrégateur pour le scénario 4 (erreur partielle)..... XI
Tableau 28: Journaux de l'agrégateur pour le scénario 5 (délai dépassé) XII
Tableau 29: Journaux de l'agrégateur pour le scénario 6 (indisponibilité d'un service)..... XIII

1. Introduction

Ce projet réalise une API Gateway d'agrégation : un client obtient, par un seul appel à Kong, la fiche d'une structure sanitaire, ses indicateurs et le résumé de ses patients. Les trois services qui détiennent ces données restent indépendants.

1.1 Contexte

Un tableau de bord de santé a besoin de données dispersées entre plusieurs systèmes : le référentiel des structures, les dossiers patients et les indicateurs d'activité. Sans passerelle, le client doit connaître chaque adresse, faire plusieurs appels et gérer seul les pannes de chacun. Fusionner ces systèmes en une seule application mélangerait des responsabilités métier distinctes.

1.2 Objectif

Développer un composant d'agrégation qui simplifie l'accès à plusieurs services sans fusionner leurs responsabilités métier, et l'exposer derrière Kong. Le système doit rester utile quand un service est lent ou en panne, et dire clairement ce qui manque.

1.3 Exigences de l'énoncé et réponse apportée

Tableau 1: TableauCorrespondance entre les exigences de l'énoncé et les réalisations du projet

N°	Exigence	Réalisation	Section
1	Trois API indépendantes	Structures, patients fictifs, indicateurs ; FastAPI, un conteneur chacune	4.1
2	Composant d'agrégation	Service Python qui appelle les 3 API en parallèle	4.2
3	Kong, point d'entrée commun	Kong 3.9 en mode DB-less, port 8000	4.3
4	GET /api/v1/dashboard/structures/{id}	Exposé par l'agrégateur, routé par Kong	4.2
5	Erreurs partielles	Statuts DISPONIBLE / ABSENT / INDISPONIBLE	5
6	Délais, corrélation, journaux	2 s / 3 s / 5 s, X-Correlation-ID, logs JSON	6
7	Tests	28 tests unitaires (dont 5 sur les contrats d'API) + 14 vérifications de bout en bout	8

2. Technologies utilisées

Le projet assemble sept briques, chacune avec un rôle précis : FastAPI pour écrire les services, httpx pour les faire dialoguer, Kong pour les exposer, Docker et Vagrant pour les exécuter, pytest pour les vérifier.

Tableau 2: Technologies utilisées et leur rôle dans le projet

Technologie	Version	Rôle dans le projet
API REST (HTTP + JSON)	—	Mode de communication : une URL, une méthode (GET, POST), une réponse JSON et un code (200, 404, 500, 503)
Python + FastAPI	Python 3.12 (conteneurs)	Écrit les 3 API et l'agrégateur ; valide les paramètres et génère la documentation Swagger sur `/docs`
Uvicorn	≥ 0.29	Serveur web asynchrone qui exécute chaque application FastAPI
httpx (asynchrone)	≥ 0.27	Client HTTP de l'agrégateur ; permet d'attendre plusieurs réponses en même temps
Kong Gateway	3.9, mode DB-less	Point d'entrée unique : routage, identifiant de corrélation, journaux, limitation de débit
Docker + Docker Compose	Docker Engine, plugin compose	Un conteneur par composant ; un fichier les démarre et les relie en réseau
Vagrant + VirtualBox	box `ubuntu/jammy64`	Crée la VM Ubuntu 22.04, installe Docker et lance la pile
pytest	≥ 8	Tests automatisés

2.1 API Gateway et agrégateur : deux rôles distincts

Une API Gateway est la porte d'entrée unique d'un ensemble de services. Elle traite les questions transversales : vers quel service envoyer la requête, comment la tracer, combien de requêtes accepter. Elle ne modifie pas le contenu métier.

Un agrégateur applique le motif API Composition : il appelle plusieurs services et assemble leurs réponses en une seule. Le placer dans un service séparé, plutôt que dans un plugin Kong, garde la logique testable en Python et laisse Kong se concentrer sur le transport.

2.2 Le mode DB-less de Kong

Kong peut stocker sa configuration dans PostgreSQL ou la lire dans un fichier YAML au démarrage. Le projet utilise le fichier kong/kong.yml : aucune base à gérer, une configuration versionnée avec le code et lisible d'un coup d'œil.

2.3 La programmation asynchrone

Avec async/await, un seul processus Python lance une requête réseau, passe à la suivante pendant l'attente, puis reprend quand la réponse arrive. L'agrégateur lance ainsi ses trois appels ensemble : si chaque service met 0,3 s, la réponse arrive en 0,3 s environ, et non en 0,9 s.

3. Architecture

Le client n'appelle qu'une adresse, le port 8000 de Kong. Kong transmet les requêtes /api/v1/dashboard à l'agrégateur, qui interroge les trois API en même temps et assemble leurs réponses.

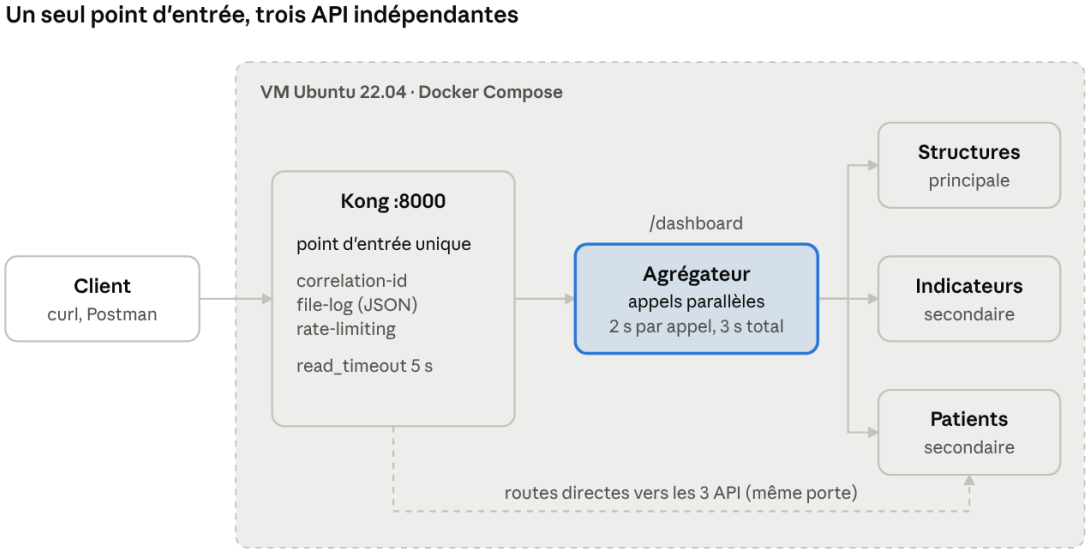


Figure 1:Architecture du système · Kong, agrégateur, 3 API

Les cinq composants tournent dans des conteneurs Docker, démarrés par Docker Compose dans une VM Ubuntu 22.04 créée par Vagrant. Kong expose aussi les trois API directement, sous leurs propres chemins, par la même porte d'entrée.

L'API structures est la source principale : sans elle, la réponse ne peut pas être construite. Les API indicateurs et patients sont des sources secondaires : leur absence donne une réponse partielle (section 5).

4. Conception et réalisation

Chaque composant a une seule responsabilité et ses propres fichiers. Le code commun aux quatre services Python (journaux, corrélation, simulation de pannes) est regroupé dans common/observability.py.

4.1 Les trois API indépendantes

Aucune API n'appelle les autres. Chacune a sa propre base de test, un fichier JSON du dossier data/ monté en volume dans son conteneur, et tourne dans son propre conteneur. Les trois bases sont créées et vérifiées par le script scripts/init_data.py.

Tableau 3: Endpoints et jeux de données des trois API indépendantes

API	Endpoints	Données
Structures sanitaires	GET /api/v1/structures (filtres region, type) · GET /api/v1/structures/{id}	5 structures : CHU Yalgado Ouédraogo, CHR Koudougou, CMA Pissy, CSPS Tanghin, CSPS Saaba
Patients fictifs	GET /api/v1/patients · GET /api/v1/patients/{id} · GET /api/v1/patients/stats?structure_id=	58 patients générés par scripts/init_data.py (graine aléatoire fixe)
Indicateurs agrégés	GET /api/v1/indicateurs · GET /api/v1/indicateurs/structures/{id}?periode=	Indicateurs mensuels d'août et septembre 2026 pour les structures 1 à 4

Trois cas de données ont été placés volontairement pour tester la distinction entre données nulles et indisponibles :

- **Structure 4 (CSPS de Tanghin)** : deces = 0, un zéro réel. taux_occupation_lits = null car un CSPS n'a pas de lits ; l'API ajoute le champ valeurs_nulles qui donne la raison NON_APPLICABLE.
- **Structure 5 (CSPS de Saaba, ouverture récente)** : aucun patient, donc total = 0 ; aucun indicateur publié, donc l'API indicateurs répond 404 INDICATORS_NOT_FOUND.
- **Structure 999** : n'existe pas ; l'API structures répond 404 STRUCTURE_NOT_FOUND.

Chaque API expose aussi deux routes techniques : GET /health, utilisée par Docker pour savoir si le conteneur est en bonne santé, et POST /_chaos, qui fait volontairement répondre le service lentement (slow) ou en erreur 500 (error). Cette seconde route sert uniquement aux tests et à la démonstration.

4.2 Le composant d'agrégation

L'agrégateur (aggregator/main.py) expose GET /api/v1/dashboard/structures/{id}. Il ne contient aucune règle métier : il orchestre les appels et compose la réponse.

Traitement d'une requête :

- Lire l'en-tête X-Correlation-ID posé par Kong, ou en générer un.
- Lancer trois tâches simultanées avec asyncio.create_task : la structure, ses indicateurs et les statistiques de ses patients. Chaque appel transmet l'identifiant de corrélation.
- Traduire chaque issue en un résultat décrit grâce à la fonction call_upstream, qui ne lève jamais d'exception : statut, données, erreur, code HTTP, durée.
- Attendre au plus 3 secondes avec asyncio.wait ; une tâche encore en cours est annulée et marquée TIMEOUT.
- Composer la réponse, choisir le code HTTP, ajouter l'en-tête X-Aggregation-Status et écrire le journal aggregation_completed.

Exemple de réponse complète (extrait) :


```
{
 "structure_id": 1,
 "statut_agregation": "COMPLET",
 "correlation_id": "8c796dec-5a7d-4a08-9162-a37aec6fdb70",
 "structure": { "statut": "DISPONIBLE", "donnees": { "code": "CHU-YO", "capacite_lits": 750 } },
 "indicateurs": { "statut": "DISPONIBLE", "donnees": { "periode": "2026-09", "consultations": 9655 } },
 "patients": { "statut": "DISPONIBLE", "donnees": { "total": 25 } },
 "sources": [
```

```
    { "service": "structures", "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 31.4 },
    { "service": "indicateurs", "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 32.0 },
    { "service": "patients", "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 20.9 }
  ]
}
```

Le bloc sources sert au diagnostic : il indique, pour chaque service, son statut, son code HTTP et sa durée de réponse.

L'agrégateur appelle les API directement par le réseau Docker, sans repasser par Kong. Kong gère le trafic qui entre dans le système (nord-sud) ; les appels internes entre services (est-ouest) n'ont pas besoin de ce détour.

4.3 La configuration de Kong

Kong écoute sur le port 8000, seule adresse connue du client. Sa configuration tient dans kong/kong.yml et repose sur trois notions : un **service** (une API en aval et ses délais), une **route** (la règle qui envoie une requête vers un service selon son chemin et sa méthode) et un **plugin** (un comportement ajouté au trafic).

Tableau 4: Routage et délais configurés dans Kong

Chemin public (port 8000)	Service Kong	Conteneur cible	read_timeout	Nouvelles tentatives
/api/v1/dashboard	aggregator-service	aggregator:8000	5 000 ms	0
/api/v1/structures	structures-service	structures:8000	3 000 ms	1
/api/v1/patients	patients-service	patients:8000	3 000 ms	1
/api/v1/indicateurs	indicateurs-service	indicateurs:8000	3 000 ms	1

Toutes les routes acceptent GET avec strip_path: false : le chemin arrive intact au service.

Tableau 5: Plugins Kong : portée et effets

Plugin	Portée	Effet
correlation-id	Global	Crée un UUID dans X-Correlation-ID si le client n'en fournit pas, le transmet au service et le renvoie au client
file-log	Global	Écrit une ligne JSON par requête : route, service, statut, latences, en-têtes
rate-limiting	Route dashboard	120 requêtes par minute ; au-delà, Kong répond 429

Aucune nouvelle tentative n'est configurée vers l'agrégateur : il gère déjà les pannes et répond toujours. Relancer la requête doublerait l'attente du client sans rien apporter.

5. Stratégie de gestion des erreurs partielles

Chaque section de la réponse (structure, indicateurs, patients) porte un statut. C'est ce statut, et non une valeur null, qui indique si une donnée est réellement vide ou simplement inconnue.

5.1 Trois statuts pour lever l'ambiguïté

Tableau 6: Les trois statuts de réponse et leur signification

Statut	Signification	Champ donnees	Exemple
DISPONIBLE	Le service a répondu	Sa réponse ; un 0 ou un null à l'intérieur est une vraie valeur	Structure 4 : deces = 0 ; taux_occupation_lits = null avec la raison NON_APPLICABLE
ABSENT	Le service a répondu qu'il n'existe rien (HTTP 404)	null, accompagné d'un motif	Structure 5 : aucun indicateur publié
INDISPONIBLE	Le service n'a pas pu répondre	null, accompagné d'un erreur.code	API indicateurs arrêtée

Sur un tableau de bord de santé, « zéro décès » et « nombre de décès inconnu » ne doivent jamais s'afficher de la même façon. Le premier est un résultat ; le second invite à réessayer plus tard.

5.2 Les causes d'indisponibilité

Tableau 7: Codes d'erreur d'indisponibilité et modes de détection par l'agrégateur

Code d'erreur	Cause	Comment l'agrégateur la détecte
TIMEOUT	Pas de réponse en 2 s, ou budget global de 3 s dépassé	asyncio.wait_for et asyncio.wait
SERVICE_UNREACHABLE	Connexion refusée, conteneur arrêté, nom introuvable	Exception httpx.TransportError
UPSTREAM_ERROR	Code 5xx ou autre code inattendu	Code HTTP différent de 200 et de 404

Code d'erreur	Cause	Comment l'agrégateur la détecte
INVALID_RESPONSE	Corps de réponse qui n'est pas du JSON	ValueError au décodage

5.3 Source principale et sources secondaires

La structure est la donnée **principale** : sans elle, on ne peut même pas confirmer que l'identifiant existe. Les indicateurs et les patients sont **secondaires** : la fiche reste utile sans eux.

Tableau 8: Règles de décision : situation des sources, code HTTP et statut d'agrégation

Situation	Code HTTP	statut_agregation
Les trois sections sont DISPONIBLE ou ABSENT	200	COMPLET
Indicateurs et/ou patients INDISPONIBLE	200	PARTIEL
Structure ABSENT (identifiant inconnu)	404	ECHEC
Structure INDISPONIBLE	503	ECHEC

Une réponse partielle garde le code 200 : la requête a abouti et apporte des données utiles. Le code 206 n'est pas adapté, car il est réservé aux requêtes Range (téléchargement par morceaux). Le client lit statut_agregation, ou l'en-tête X-Aggregation-Status, pour savoir s'il doit signaler un manque. Même en 503, le corps contient ce qui a pu être récupéré, pour faciliter le diagnostic.

6. Délais d'attente, corrélation et journaux structurés

Ces trois mécanismes rendent le système prévisible quand un service ralentit, et traçable quand il faut comprendre ce qui s'est passé.

6.1 Des délais emboités : 2 s, 3 s, 5 s

Chaque couche accorde plus de temps que la couche qu'elle appelle. C'est ainsi toujours la couche la mieux informée qui répond en premier, avec un message précis.

Tableau 9: Délais d'attente emboîtés et comportement en cas de dépassement

Couche	Délai	Où il est réglé	Effet s'il est dépassé
Appel d'une API par l'agrégateur	2 s	UPSTREAM_TIMEOUT dans docker-compose.yml	La section passe en INDISPONIBLE / TIMEOUT ; les autres sont servies

Couche	Délai	Où il est réglé	Effet s'il est dépassé
Agrégation complète	3 s	GLOBAL_TIMEOUT dans docker-compose.yml	Les tâches encore en cours sont annulées et marquées TIMEOUT
Kong vers l'agrégateur	5 s	read_timeout: 5000 dans kong/kong.yml	Kong répond 504 sans détail ; filet de sécurité qui ne devrait jamais servir

Si l'ordre était inversé (Kong à 2 s, API à 5 s), Kong couperait la connexion avant l'agrégateur. Le client recevrait un 504 générique et perdrait les sections déjà obtenues.

6.2 L'identifiant de corrélation

Une requête du client traverse cinq composants. Sans identifiant commun, il est impossible de relier leurs journaux. L'en-tête X-Correlation-ID suit ce parcours :

1. Kong le crée avec le plugin correlation-id, ou conserve celui envoyé par le client.
2. Le middleware de l'agrégateur le range dans une ContextVar Python, une variable propre à chaque requête.
3. L'agrégateur l'ajoute à chacun des trois appels sortants.
4. Chaque API l'inscrit dans ses journaux et le renvoie dans sa réponse.
5. Le client le reçoit dans l'en-tête de réponse et dans le champ correlation_id du corps.

Chaque tâche asyncio copie le contexte de la requête qui l'a créée. Les trois appels parallèles voient donc le même identifiant, et deux requêtes simultanées ne se mélangent jamais.

6.3 Les journaux structurés

Chaque service écrit une ligne JSON par événement au lieu d'une phrase libre. Une machine peut ainsi filtrer et compter ces lignes avec grep ou jq, ou dans un outil centralisé comme ELK ou Loki en production.

{"timestamp": "2026-10-19T12:01:29.912+00:00", "level": "INFO", "service": "aggregator", "correlation_id": "test-123", "event": "upstream_call", "upstream": "indicateurs", "outcome": "DISPONIBLE", "http_status": 200, "error_code": null, "duration_ms": 32.0}

Tableau 10: Types d'événements journalisés et informations associées

Événement	Émis par	Informations utiles
http_request	Les 4 services Python	Méthode, chemin, code de réponse, durée
upstream_call	Agrégateur, une ligne par appel	Service appelé, issue, code d'erreur, durée ; niveau WARNING si indisponible
aggregation_completed	Agrégateur	Statut final, code HTTP, statut de chaque source, durée totale
chaos_mode_changed	Service concerné	Mode de panne et délai activés
Journal file-log	Kong	Route, service, latences de Kong et de l'amont, en-têtes

7. Déploiement

Toute la pile démarre avec une seule commande, vagrant up, sur n'importe quel poste disposant de VirtualBox et de Vagrant. Trois fichiers décrivent cette infrastructure.

7.1 Le Dockerfile

Une seule image sert aux quatre services Python. Le service à embarquer est choisi au moment de la construction par l'argument SERVICE_DIR. L'image part de python:3.12-slim, installe les dépendances de requirements.txt, copie le dossier common et le service choisi, puis lance Uvicorn sur le port 8000. Une instruction HEALTHCHECK interroge /health toutes les 10 secondes.

7.2 Le fichier docker-compose.yml

Il déclare les cinq conteneurs et les relie dans un même réseau, où chacun est joignable par son nom (structures, aggregator…).

Tableau 11: Conteneurs Docker, images et ports exposés sur la machine virtuelle

Conteneur	Image	Port sur la VM	Rôle de ce port
kong	kong:3.9	8000 et 8001	8000 : point d'entrée des clients ; 8001 : API d'administration de Kong
aggregator	construite	9000	Accès direct et documentation Swagger
structures-api	construite	9001	Accès direct pour simuler des pannes

Conteneur	Image	Port sur la VM	Rôle de ce port
patients-api	construite	9002	Idem
indicateurs-api	construite	9003	Idem

L'agrégateur reçoit les adresses des API et ses délais par variables d'environnement. Il ne dépend pas du démarrage des API : il doit pouvoir répondre partiellement même si l'une d'elles est arrêtée.

7.3 Le Vagrantfile

Il crée une VM Ubuntu 22.04 (box ubuntu/jammy64, 3 Go de mémoire, 2 processeurs). Au premier démarrage, un script installe Docker, jq et les dépendances de test, puis lance docker compose up -d --build. Les ports 8000, 8001 et 9000 de la VM sont redirigés vers la machine hôte ; les ports 9001 à 9003 restent internes à la VM.

Le dossier du projet est partagé avec la VM sous /vagrant : une modification faite sous Windows est visible immédiatement dans la VM.

7.4 Dépôt Git et installation reproductible

Le projet est livré dans un dépôt Git public, installable sur une machine neuve en suivant le README.md. Les choix suivants garantissent que l'installation donne le même résultat partout.

Tableau 12: Dépôt Git : éléments qui rendent l'installation reproductible

Élément	Emplacement	Rôle
Bases de test	data/*.json	Montées en lecture seule dans chaque conteneur (volumes Docker)
Création des bases	scripts/init_data.py	Recrée les 3 bases (graine aléatoire fixe) et vérifie leur cohérence ; --check vérifie sans écrire
Contrats d'API	contracts/*.openapi.json	Contrats OpenAPI 3 des 4 services, exportés par scripts/export_openapi.py
Workflow	.github/workflows/ci.yml	À chaque push : tests unitaires, puis pile complète et tests de bout en bout
Dépendances	requirements.txt	Versions figées (FastAPI 0.147.0, Uvicorn 0.53.0, httpx 0.28.1…)

Élément	Emplacement	Rôle
Redémarrage	docker-compose.yml	restart: unless-stopped : les conteneurs reviennent seuls après un redémarrage
Procédures	docs/INSTALLATION.md, docs/RESULTATS_ATTENDUS.md	Installation des outils (Docker, VirtualBox, Vagrant, Git) et résultats attendus des tests

Deux installations sont documentées : avec Docker Compose sur une machine qui a déjà Docker, ou avec vagrant up sur une machine neuve qui n'a que VirtualBox et Vagrant.

8. Tests et résultats

La suite compte aujourd'hui 28 tests unitaires et 14 vérifications de bout en bout. Lors de la démonstration du 21 octobre, avant l'ajout des 5 tests de contrats, les deux niveaux ont réussi dans la VM Vagrant, avec les cinq conteneurs en état healthy : 23 tests unitaires sur 23 et 14 vérifications de bout en bout sur 14, à travers Kong.

8.1 Tests unitaires (pytest)

Commande : pytest -v tests/. Aucun service n'a besoin d'être démarré.

Pour tester l'agrégateur, les trois API sont remplacées par un faux transport HTTP (httpx.MockTransport). Chaque test décide si un service répond normalement, répond 404 ou 500, met 5 s, est arrêté ou renvoie du HTML. Les délais sont réduits à 0,5 s et 1 s pour que la suite s'exécute en quelques secondes. Le fichier tests/test_services.py teste en plus chaque API isolément, et tests/test_contracts.py (5 tests) vérifie que les contrats publiés dans contracts/ correspondent exactement au code.

8.2 Tests de bout en bout

Commande : bash tests/e2e_kong.sh, pile Docker démarrée. Le script envoie ses requêtes à Kong sur le port 8000, provoque les pannes avec /_chaos et docker compose stop, puis remet chaque service dans son état normal.

8.3 Couverture des exigences

Tableau 13: Couverture des exigences par les tests unitaires et les vérifications de bout en bout

Exigence testée	Tests unitaires	Bout en bout	Résultat
Réponse complète	test_reponse_complete	Scénario 1 : 200 COMPLET	Réussi
Données réellement nulles	test_valeurs_reellement_nulles_conservees, test_indicateurs_absents_ne_sont_pas_une_panne	Scénario 2 : total = 0, indicateurs ABSENT	Réussi
Structure inconnue	test_structure_inexistante_404	Scénario 3 : 404	Réussi
Erreur partielle	test_erreur_partielle_service_en_erreur_500, test_reponse_invalide	Scénario 4 : 200 PARTIEL, UPSTREAM_ERROR	Réussi
Délai dépassé	test_delai_depasse, test_budget_global, test_appels_en_parallele	Scénario 5 : TIMEOUT en moins de 3 s	Réussi
Service indisponible	test_service_secondaire_injoignable, test_tous_les_secondaires_indisponibles	Scénario 6 : PARTIEL, SERVICE_UNREACHABLE	Réussi
Service principal indisponible	test_service_principal_indisponible_503	Scénario 7 : 503	Réussi
Identifiant de corrélation	test_correlation_id_propage_aux_trois_services	Scénario 8 : en-tête créé par Kong	Réussi

Sortie obtenue dans la VM pour les tests unitaires (résumé) :

```
platform linux -- Python 3.10.12, pytest-9.1.1
collected 23 items
...
==================== 23 passed, 1 warning in 5.25s ====================
```

L'avertissement signale un changement à venir dans la bibliothèque de test de Starlette ; il est sans effet sur les résultats.

9. Guide de démonstration — préparer l'environnement

La préparation prend environ 2 minutes si la VM existe déjà. Toutes les commandes de la démonstration se tapent dans la VM, pas dans PowerShell : les ports 9001 à 9003, nécessaires pour simuler les pannes, ne sont joignables que depuis la VM.

9.1 Démarrer la VM et s'y connecter

Dans PowerShell, sur Windows :

```
cd C:\Users\LENOVO\Downloads\MASTER1\service_web\TP\api-gateway-sante
vagrant up
vagrant ssh
```

Tableau 14: Commandes de démarrage et de connexion à la machine virtuelle

Commande	Rôle	Objectif
cd <dossier du projet>	Se placer dans le dossier qui contient le Vagrantfile	Vagrant cherche ce fichier dans le dossier courant
vagrant up	Démarre la VM ; la crée et l'installe au tout premier lancement	Avoir une machine Ubuntu identique à celle des tests
vagrant ssh	Ouvre une session en ligne de commande dans la VM	Travailler dans l'environnement où tournent les conteneurs

Un vagrant up sur une VM déjà créée ne relance pas le script d'installation, donc ne démarre pas les conteneurs. L'étape 9.2 s'en charge.

9.2 Démarrer et vérifier la pile

Dans la VM :

```
cd /vagrant
docker compose up -d
docker compose ps
bash tests/e2e_kong.sh
```

Tableau 15: Commandes de démarrage et de vérification de la pile Docker

Commande	Rôle	Objectif
cd /vagrant	Entrer dans le dossier du projet, partagé avec Windows	docker compose lit docker-compose.yml dans le dossier courant
docker compose up -d	Démarre les 5 conteneurs ; -d (*detached*) les lance en arrière-plan et rend la main	Remettre la pile en marche après un redémarrage de la VM

Commande	Rôle	Objectif
docker compose ps	Liste les conteneurs et leur état	Attendre que les 5 soient Up (healthy), soit environ 30 s
bash tests/e2e_kong.sh	Exécute les 14 vérifications de bout en bout	S'assurer, juste avant de passer, que tout fonctionne ; le script remet tout en état normal à la fin

9.3 Ouvrir deux terminaux

Ouvrez une seconde fenêtre PowerShell (ou divisez Windows Terminal avec **Alt + Maj + D**), puis refaites cd <dossier du projet>, vagrant ssh et cd /vagrant. Chaque vagrant ssh ouvre une session de plus dans la même VM.

Terminal 1 — les journaux, affichés au jury pendant toute la démo :

```
docker compose logs -f --tail 0 aggregator
```

Tableau 16: Rôle des éléments de la commande de suivi des journaux

Élément	Rôle
docker compose logs	Affiche ce que les conteneurs écrivent sur leur sortie standard, ici nos journaux JSON
-f (*follow*)	Continue d'afficher les nouvelles lignes au fil de l'eau, sans rendre la main
--tail 0	N'affiche aucune ligne ancienne : l'écran commence vide
aggregator	Limite l'affichage à l'agrégateur

Objectif : montrer en direct, pour chaque requête, les trois appels de l'agrégateur, leur issue et le statut final. On se limite à l'agrégateur parce que les lignes de Kong, très longues, rendraient l'écran illisible. Ctrl + C arrête l'affichage sans arrêter le conteneur.

Terminal 2 — les requêtes : toutes les commandes de la section 10.

9.4 Les outils utilisés dans les commandes

Tableau 17: Outils et options utilisés dans les commandes de démonstration

Outil ou option	Rôle
curl	Client HTTP en ligne de commande : envoie une requête et affiche la réponse
-s (*silent*)	Masque la barre de progression de curl, pour ne garder que la réponse
-i (*include*)	Affiche aussi la ligne de statut (HTTP/1.1 200 OK) et les en-têtes de réponse
-X POST	Utilise la méthode POST au lieu de GET (par défaut)
-H 'Nom: valeur'	Ajoute un en-tête à la requête
-d '...'	Envoie un corps de requête (ici du JSON)
| (barre verticale)	Passe la sortie d'une commande en entrée de la suivante
jq	Met en forme le JSON et en extrait des parties
jq '{a, b}'	Ne garde que les champs a et b de la réponse
jq '.x.y'	Descend dans le champ x puis dans son sous-champ y
localhost	La machine sur laquelle on tape la commande, ici la VM
:8000, :8001, :9002…	Le port visé : 8000 = Kong, 8001 = administration de Kong, 9001 à 9003 = accès direct aux API

10. Guide de démonstration — les 7 scénarios

La démonstration dure environ 8 minutes. Chaque scénario prouve une exigence de l'énoncé ; il se déroule toujours de la même façon : provoquer une situation, interroger le tableau de bord, montrer la réponse dans le terminal 2 et les journaux dans le terminal 1, puis remettre le système en état normal. Les sorties réellement obtenues pour chaque scénario figurent en annexe B.

Scénario 1 — Kong est le point d'entrée unique

Objectif : prouver que Kong connaît les quatre services et que sa configuration est bien chargée (exigence 3).

```
curl -s localhost:8001/services | jq '.data[].name'
```

Tableau 18: Éléments de la commande de vérification des services déclarés dans Kong

Élément	Rôle
localhost:8001/services	Interroge l'API d'administration de Kong, qui liste les services déclarés
jq '.data[].name'	Parcourt le tableau data de la réponse et n'affiche que le champ name de chaque service

Résultat attendu : "aggregator-service", "indicateurs-service", "patients-service", "structures-service".

À dire au jury : « Le client ne connaît qu'une adresse, le port 8000 de Kong. Ces quatre services viennent du fichier kong.yml, chargé au démarrage sans base de données. »

Scénario 2 — Réponse complète

Objectif : montrer l'endpoint demandé qui rassemble trois services en une réponse (exigences 2 et 4).

```
curl -s localhost:8000/api/v1/dashboard/structures/1 | jq
```

Tableau 19: Éléments de la commande d'appel au tableau de bord via Kong

Élément	Rôle
localhost:8000	Passe par Kong, comme le ferait un vrai client
/api/v1/dashboard/structures/1	L'endpoint de l'énoncé, pour la structure 1 (CHU Yalgado Ouédraogo)
jq sans filtre	Affiche toute la réponse, indentée et en couleur

Résultat attendu : statut_agregation: "COMPLET", les trois sections DISPONIBLE, et le bloc sources avec trois codes 200. Dans le terminal 1 : trois lignes upstream_call puis une ligne aggregation_completed.

À dire au jury : « Un seul appel, trois services interrogés en parallèle. Le bloc sources indique qui a répondu et en combien de millisecondes. »

Scénario 3 — Des données réellement nulles

Objectif : montrer qu'un zéro ou une absence de données n'est pas confondu avec une panne (exigence 5).

```
curl -s localhost:8000/api/v1/dashboard/structures/4 | jq '.indicateurs'
curl -s localhost:8000/api/v1/dashboard/structures/5 | jq '{statut_agregation, total_patients: .patients.donnees.total, indicateurs}'
```

Tableau 20: Éléments des commandes d'extraction des données du scénario 3

Élément	Rôle
jq '.indicateurs'	N'affiche que la section indicateurs de la structure 4
jq '{statut_agregation, …}'	Construit un petit objet avec le statut global, le nombre de patients et la section indicateurs
total_patients: .patients.donnees.total	Renomme le champ extrait pour le rendre lisible

Résultat attendu :

- Structure 4 : deces: 0 et taux_occupation_lits: null, avec dans valeurs_nulles la raison NON_APPLICABLE.
- Structure 5 : COMPLET, total_patients: 0, indicateurs ABSENT avec le motif INDICATORS_NOT_FOUND.

À dire au jury : « Zéro décès est une vraie valeur. Un CSPS n'a pas de lits, donc le taux d'occupation est non applicable, et l'API le dit. La structure 5 n'a encore rien publié : c'est ABSENT, la réponse reste COMPLÈTE. Rien de tout cela n'est une panne. »

Scénario 4 — Erreur partielle

Objectif : montrer qu'une API en erreur n'empêche pas de servir le reste (exigences 5 et 7).

```
curl -s -X POST localhost:9002/_chaos -H 'Content-Type: application/json' -d '{"mode":"error"}'
curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, patients}'
curl -s -X POST localhost:9002/_chaos -H 'Content-Type: application/json' -d '{"mode":"normal"}'
```

Tableau 21: Rôle des commandes utilisées dans le scénario 4

Commande	Rôle
1re ligne	Envoie directement à l'API patients (port 9002) l'ordre de répondre 500 à toutes ses requêtes métier. -X POST choisit la méthode, -H déclare un corps JSON, -d contient ce corps

Commande	Rôle
2e ligne	Interroge le tableau de bord via Kong et n'affiche que le statut global et la section patients
3e ligne	Remet l'API patients en fonctionnement normal

Résultat attendu : HTTP 200, statut_agregation: "PARTIEL", patients INDISPONIBLE avec erreur.code: "UPSTREAM_ERROR". Dans le terminal 1, l'appel aux patients apparaît en WARNING.

À dire au jury : « L'API patients est en panne, mais la fiche et les indicateurs sont servis. La réponse dit PARTIEL et précise ce qui manque et pourquoi. »

Scénario 5 — Délai dépassé

Objectif : montrer que l'agrégateur n'attend pas un service lent au-delà de 2 secondes (exigence 6).

```
curl -s -X POST localhost:9003/_chaos -H 'Content-Type: application/json' -d '{"mode":"slow","delay":5}'
time curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, indicateurs}'
curl -s -X POST localhost:9003/_chaos -H 'Content-Type: application/json' -d '{"mode":"normal"}'
```

Tableau 22: Rôle des commandes utilisées dans le scénario 5

Commande	Rôle
1re ligne	Demande à l'API indicateurs (port 9003) d'attendre 5 s avant chaque réponse
time devant la 2e ligne	Mesure la durée réelle de la commande et l'affiche à la fin (real)
3e ligne	Remet l'API indicateurs en fonctionnement normal

Résultat attendu : indicateurs INDISPONIBLE avec erreur.code: "TIMEOUT", et real 0m2.0…s.

À dire au jury : « Le service met 5 secondes, mais l'utilisateur attend 2 secondes seulement. Les délais sont emboités : 2 s par appel, 3 s pour l'agrégation, 5 s dans Kong. C'est toujours la couche la mieux informée qui répond. »

Scénario 6 — Indisponibilité d'un service

Objectif : distinguer la panne d'une source secondaire de celle de la source principale (exigences 5 et 7).

```
docker compose stop indicateurs
curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, indicateurs}'
```

```
docker compose start indicateurs

docker compose stop structures
curl -si localhost:8000/api/v1/dashboard/structures/1 | head -n 1
curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, erreur}'
docker compose start structures
```

Tableau 23: Rôle des commandes utilisées dans le scénario 6

Commande	Rôle
`docker compose stop <service>`	Arrête réellement le conteneur : c'est une vraie panne, pas une simulation
`curl -si … | head -n 1`	`-i` affiche la ligne de statut HTTP ; `head -n 1` ne garde que cette première ligne
`docker compose start <service>`	Redémarre le conteneur ; attendre environ 5 s avant la commande suivante

Résultat attendu :

- Indicateurs arrêtés : 200, `PARTIEL`, `erreur.code: "SERVICE_UNREACHABLE"`.
- Structures arrêtée : `HTTP/1.1 503 Service Unavailable`, `ECHEC`, `erreur.code: "PRIMARY_SOURCE_UNAVAILABLE"`.

À dire au jury : « Sans les indicateurs, la fiche reste utile : réponse partielle. Sans la structure, on ne peut même pas confirmer que l'identifiant existe : 503. Même dans ce cas, le corps contient ce qui a pu être récupéré. »

Scénario 7 — Suivre une requête grâce à l'identifiant de corrélation

Objectif : prouver qu'on peut retrouver une même requête dans les journaux des cinq composants (exigence 6).

```
curl -si localhost:8000/api/v1/dashboard/structures/1 | grep -i x-correlation-id
curl -s -H "X-Correlation-ID: demo-soutenance" localhost:8000/api/v1/dashboard/structures/1 > /dev/null
docker compose logs | grep demo-soutenance
```

Tableau 24: Rôle des commandes utilisées dans le scénario 7

Commande	Rôle
1re ligne	Affiche les en-têtes (`-i`) et ne garde, avec `grep -i`, que la ligne X-Correlation-ID ; `-i` de grep ignore la casse. Montre l'identifiant généré par Kong

Commande	Rôle
2e ligne	Envoie un identifiant choisi, facile à taper ; Kong le conserve. Le jour de la soutenance, choisissez-en un jamais utilisé (par exemple demo-jury) pour ne pas retrouver les traces des répétitions. > /dev/null jette la réponse, inutile ici
3e ligne	Affiche les journaux de tous les conteneurs et ne garde que les lignes qui contiennent cet identifiant

Résultat attendu : des lignes préfixées par kong, aggregator, structures-api, indicateurs-api et patients-api, toutes avec demo-soutenance.

À dire au jury : « Une seule commande retrace le parcours complet de la requête à travers les cinq composants. En production, c'est ainsi qu'on diagnostique un incident. »

En cas de problème pendant la démo

Tableau 25: Guide de dépannage pendant la démonstration

Symptôme	Cause probable	Correction
curl: (7) Failed to connect	Les conteneurs ne sont pas démarrés	docker compose up -d, puis docker compose ps
Réponse toujours PARTIEL	Un mode chaos est resté actif	Envoyer {"mode":"normal"} aux ports 9001, 9002 et 9003
503 après un start	Le service redémarre encore	Attendre 5 s et relancer la requête
jq: command not found	jq absent de la VM	sudo apt-get install -y jq

11. Conclusion et perspectives

Le projet répond aux sept exigences de l'énoncé : le client obtient en un appel à Kong la fiche, les indicateurs et le résumé des patients d'une structure, et la panne d'un service secondaire donne une réponse partielle explicite au lieu d'une erreur. Les 28 tests unitaires et les 14 vérifications de bout en bout le confirment, et le workflow d'intégration continue les rejoue à chaque modification du dépôt.

Le principal enseignement concerne la séparation des rôles. Kong traite le transport (routage, traçage, limitation de débit), l'agrégateur traite la composition, et chaque API garde son métier. La

distinction entre DISPONIBLE, ABSENT et INDISPONIBLE est ce qui rend le tableau de bord fiable : un zéro n'y est jamais confondu avec une donnée manquante.

Pistes d'amélioration pour un usage réel :

- **Sécurité** : authentification JWT ou clé d'API sur Kong (plugins `jwt`, `key-auth`), HTTPS, contrôle d'accès, indispensables pour des données de santé.
- **Performance** : cache court sur le tableau de bord (plugin `proxy-cache`) et plusieurs instances de l'agrégateur, qui ne conserve aucun état.
- **Résilience** : disjoncteur (*circuit breaker*) pour ne plus appeler pendant quelques secondes un service qui échoue, et dernières valeurs connues servies avec la mention de leur ancienneté.
- **Supervision** : métriques Prometheus et tableaux Grafana, centralisation des journaux JSON dans ELK ou Loki.

ANNEXES

Annexe A — Arborescence du projet

```
api-gateway-sante/
├── aggregator/main.py            composant d'agrégation (endpoint dashboard)
├── services/
│   ├── structures/main.py        API 1 — structures sanitaires
│   ├── patients/main.py          API 2 — patients fictifs
│   └── indicateurs/main.py       API 3 — indicateurs agrégés
├── common/
│   ├── observability.py          journaux JSON, corrélation, /health, mode chaos
│   └── data.py                   chargement des bases de test
├── data/                         bases de test JSON (volumes Docker)
├── scripts/
│   ├── init_data.py              création et initialisation des bases
│   └── export_openapi.py         export des contrats d'API
├── contracts/                    contrats OpenAPI 3 des 4 services
├── kong/kong.yml                 configuration déclarative de Kong
├── tests/
│   ├── test_services.py          10 tests des API
│   ├── test_aggregator.py        13 tests de l'agrégateur
│   ├── test_contracts.py         5 tests des contrats d'API
│   └── e2e_kong.sh               14 vérifications à travers Kong
├── .github/workflows/ci.yml      intégration continue (GitHub Actions)
├── docs/                         INSTALLATION.md, RESULTATS_ATTENDUS.md,
                                  présentation (.pptx), rapport (.pdf)
├── Dockerfile                    image commune des 4 services Python
├── docker-compose.yml            les 5 conteneurs, leurs volumes et leur réseau
├── Vagrantfile                   VM Ubuntu 22.04 + installation automatique
├── requirements.txt / requirements-dev.txt   dépendances (versions figées)
└── README.md                     architecture, prérequis, installation
```

Annexe B — Sorties obtenues lors de la démonstration

La démonstration a été exécutée le 21 octobre 2026 dans la VM Vagrant : les 7 scénarios ont donné le résultat attendu. Les sorties du terminal 2 sont reproduites telles quelles. Les journaux du terminal 1 sont présentés en tableau, avec les champs utiles de chaque ligne JSON.

B.1 Scénario 1 — Services déclarés dans Kong

```
$ curl -s localhost:8001/services | jq '.data[].name'
"structures-service"
"indicateurs-service"
"patients-service"
"aggregator-service"
```

B.2 Scénario 2 — Réponse complète

```
$ curl -s localhost:8000/api/v1/dashboard/structures/1 | jq
{
 "structure_id": 1,
 "statut_agregation": "COMPLET",
 "correlation_id": "4a561e93-75e2-42a5-b85b-f8442db177da",
 "genere_le": "2026-10-21T12:13:46+00:00",
 "structure": {
  "statut": "DISPONIBLE",
  "donnees": {
   "id": 1, "code": "CHU-YO",
   "nom": "Centre Hospitalier Universitaire Yalgado Ouédraogo",
   "type": "CHU", "region": "Centre", "district": "Baskuy", "ville": "Ouagadougou",
   "capacite_lits": 750, "telephone": "+226 25 00 00 01", "statut": "public"
  }
 },
 "indicateurs": {
  "statut": "DISPONIBLE",
  "donnees": {
   "structure_id": 1, "periode": "2026-09",
   "consultations": 9655, "hospitalisations": 1912, "accouchements": 428,
   "deces": 41, "cas_paludisme": 2540, "taux_occupation_lits": 89.2,
   "duree_moyenne_sejour_jours": 6.4, "taux_couverture_vaccinale": null,
   "valeurs_nulles": {
    "taux_couverture_vaccinale": "NON_APPLICABLE: vaccination non réalisée dans cette structure"
   }
  }
 },
 "patients": {
  "statut": "DISPONIBLE",
  "donnees": {
   "structure_id": 1, "total": 25,
   "par_sexe": { "F": 11, "M": 14 },
   "par_statut": { "ambulatoire": 2, "hospitalise": 12, "sorti": 11 },
   "par_tranche_age": { "0-4": 0, "5-14": 3, "15-49": 8, "50+": 14 }
  }
 },
 "sources": [
  { "service": "structures",  "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 32.6 },
  { "service": "indicateurs", "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 39 },
  { "service": "patients",    "statut": "DISPONIBLE", "http_status": 200, "duree_ms": 32.8 }
 ]
}
```

Pour la lisibilité, les champs d'un même objet sont regroupés sur moins de lignes ; leur contenu est inchangé.

Terminal 1 (corrélation `4a561e93-…`) :

Tableau 26: Journaux de l'agrégateur pour le scénario 2 (réponse complète)

Événement	Service appelé	Issue	Code HTTP	Durée
upstream_call	structures	DISPONIBLE	200	32,6 ms
upstream_call	patients	DISPONIBLE	200	32,8 ms
upstream_call	indicateurs	DISPONIBLE	200	39,0 ms
aggregation_completed	—	COMPLET	200	42,1 ms

Les trois appels se terminent en 32 à 39 ms et l'agrégation en 42 ms : la durée totale est celle de l'appel le plus lent, et non la somme des trois (environ 104 ms). C'est la preuve que les appels sont bien parallèles.

B.3 Scénario 3 — Données réellement nulles

```
$ curl -s localhost:8000/api/v1/dashboard/structures/4 | jq '.indicateurs'
{
  "statut": "DISPONIBLE",
  "donnees": {
    "structure_id": 4,
    "periode": "2026-09",
    "consultations": 1415,
    "hospitalisations": 0,
    "accouchements": 41,
    "deces": 0,
    "cas_paludisme": 598,
    "taux_occupation_lits": null,
    "duree_moyenne_sejour_jours": null,
    "taux_couverture_vaccinale": 96,
    "valeurs_nulles": {
      "taux_occupation_lits": "NON_APPLICABLE: structure sans lits d'hospitalisation",
      "duree_moyenne_sejour_jours": "NON_APPLICABLE: structure sans hospitalisation"
    }
  }
}

$ curl -s localhost:8000/api/v1/dashboard/structures/5 | jq '{statut_agregation, total_patients:
.patients.donnees.total, indicateurs}'
{
  "statut_agregation": "COMPLET",
  "total_patients": 0,
  "indicateurs": {
    "statut": "ABSENT",
    "donnees": null,
    "motif": {
```

```
    "error": "INDICATORS_NOT_FOUND",
    "structure_id": 5
   }
  }
}
```

Terminal 1, structure 5 (corrélation `97c5d849-…`) :

Tableau 27: Journaux de l'agrégateur pour la structure 5 (scénario 3)

Événement	Service appelé	Issue	Code HTTP	Durée
upstream_call	indicateurs	ABSENT	404	38,5 ms
upstream_call	structures	DISPONIBLE	200	42,7 ms
upstream_call	patients	DISPONIBLE	200	42,1 ms
aggregation_completed	—	COMPLET	200	47,6 ms

Le 404 des indicateurs est journalisé au niveau `INFO`, et non `WARNING` : l'absence de données n'est pas traitée comme une panne.

B.4 Scénario 4 — Erreur partielle

```
$ curl -s -X POST localhost:9002/_chaos -H 'Content-Type: application/json' -d '{"mode":"error"}'
{"mode":"error","delay":5.0}

$ curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, patients}'
{
  "statut_agregation": "PARTIEL",
  "patients": {
    "statut": "INDISPONIBLE",
    "donnees": null,
    "erreur": {
      "code": "UPSTREAM_ERROR",
      "message": "Le service a répondu HTTP 500"
    }
  }
}

$ curl -s -X POST localhost:9002/_chaos -H 'Content-Type: application/json' -d '{"mode":"normal"}'
{"mode":"normal","delay":5.0}
```

Terminal 1 (corrélation `b04f1550-…`) :

Tableau 28: Journaux de l'agrégateur pour le scénario 4 (erreur partielle)

Niveau	Événement	Service appelé	Issue	Code HTTP	Erreur	Durée
WARNING	upstream_call	patients	INDISPONIBLE	500	UPSTREAM_ERROR	34,4 ms
INFO	upstream_call	indicateurs	DISPONIBLE	200	—	39,9 ms
INFO	upstream_call	structures	DISPONIBLE	200	—	47,7 ms
INFO	aggregation_completed	—	PARTIEL	200	—	49,6 ms

B.5 Scénario 5 — Délai dépassé

```
$ curl -s -X POST localhost:9003/_chaos -H 'Content-Type: application/json' -d
'{"mode":"slow","delay":5}'
{"mode":"slow","delay":5.0}

$ time curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, indicateurs}'
{
  "statut_agregation": "PARTIEL",
  "indicateurs": {
    "statut": "INDISPONIBLE",
    "donnees": null,
    "erreur": {
      "code": "TIMEOUT",
      "message": "Pas de réponse en moins de 2.0 s"
    }
  }
}

real    0m2.040s
user    0m0.069s
sys     0m0.019s

$ curl -s -X POST localhost:9003/_chaos -H 'Content-Type: application/json' -d '{"mode":"normal"}'
{"mode":"normal","delay":5.0}
```

Terminal 1 (corrélation f0a0df7c-…) :

Tableau 29: Journaux de l'agrégateur pour le scénario 5 (délai dépassé)

Niveau	Événement	Service appelé	Issue	Erreur	Durée
INFO	upstream_call	patients	DISPONIBLE	—	29,4 ms
INFO	upstream_call	structures	DISPONIBLE	—	38,7 ms
WARNING	upstream_call	indicateurs	INDISPONIBLE	TIMEOUT	2 005,4 ms
INFO	aggregation_completed	—	PARTIEL	—	2 007,6 ms

Le service indicateurs aurait mis 5 s ; l'agrégateur l'a abandonné à 2 s et le client a eu sa réponse en 2,04 s.

B.6 Scénario 6 — Indisponibilité d'un service

```
$ docker compose stop indicateurs
 ✔ Container indicateurs-api Stopped

$ curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, indicateurs}'
{
  "statut_agregation": "PARTIEL",
  "indicateurs": {
    "statut": "INDISPONIBLE",
    "donnees": null,
    "erreur": {
      "code": "SERVICE_UNREACHABLE",
      "message": "Service injoignable (ConnectError)"
    }
  }
}

$ docker compose start indicateurs
 ✔ Container indicateurs-api Started

$ docker compose stop structures
 ✔ Container structures-api Stopped

$ curl -si localhost:8000/api/v1/dashboard/structures/1 | head -n 1
HTTP/1.1 503 Service Unavailable

$ curl -s localhost:8000/api/v1/dashboard/structures/1 | jq '{statut_agregation, erreur}'
{
  "statut_agregation": "ECHEC",
```

```
  "erreur": {
    "code": "PRIMARY_SOURCE_UNAVAILABLE",
    "message": "Le service des structures est indisponible"
  }
}

$ docker compose start structures
 ✔ Container structures-api Started
```

Terminal 1 :

Tableau 30: Journaux de l'agrégateur pour le scénario 6 (indisponibilité d'un service)

Cas	Service appelé	Issue	Erreur	Durée	Résultat de l'agrégation
Indicateurs arrêtés (ba7b9909-…)	indicateurs	INDISPONIBLE	SERVICE_UNREACHABLE	56,8 ms	PARTIEL, HTTP 200
Structures arrêtée (3c7d0a88-…)	structures	INDISPONIBLE	SERVICE_UNREACHABLE	25,9 ms	ECHEC, HTTP 503

Un service arrêté est détecté en quelques dizaines de millisecondes (connexion refusée), sans attendre le délai de 2 s.

B.7 Scénario 7 — Identifiant de corrélation

```
$ curl -si localhost:8000/api/v1/dashboard/structures/1 | grep -i x-correlation-id
x-correlation-id: 5394aa65-5ba9-4270-99f8-1a961518bc4c

$ curl -s -H "X-Correlation-ID: demo-soutenance" localhost:8000/api/v1/dashboard/structures/1 >
/dev/null

$ docker compose logs | grep demo-soutenance
kong            | {"source":"upstream","correlation_id":"demo-
soutenance","latencies":{"request":51,"receive":0,"kong":0,"proxy":51}, …
,"service":{"name":"aggregator-service","read_timeout":5000,"retries":0, … },"upstream_status":"200"}
patients-api    | {"timestamp": "2026-10-21T13:09:15.847+00:00", "service": "patients-api",
"correlation_id": "demo-soutenance", "event": "http_request", "path": "/api/v1/patients/stats", "status":
```

200, "duration_ms": 1.4}
aggregator | {"timestamp": "2026-10-21T13:09:15.855+00:00", "service": "aggregator", "correlation_id": "demo-soutenance", "event": "upstream_call", "upstream": "indicateurs", "outcome": "DISPONIBLE", "duration_ms": 24.0}
aggregator | {"timestamp": "2026-10-21T13:09:15.856+00:00", "service": "aggregator", "correlation_id": "demo-soutenance", "event": "upstream_call", "upstream": "patients", "outcome": "DISPONIBLE", "duration_ms": 22.9}
aggregator | {"timestamp": "2026-10-21T13:09:15.867+00:00", "service": "aggregator", "correlation_id": "demo-soutenance", "event": "upstream_call", "upstream": "structures", "outcome": "DISPONIBLE", "duration_ms": 38.2}
aggregator | {"timestamp": "2026-10-21T13:09:15.869+00:00", "service": "aggregator", "correlation_id": "demo-soutenance", "event": "aggregation_completed", "statut_agregation": "COMPLET", "duration_ms": 40.2}
aggregator | {"timestamp": "2026-10-21T13:09:15.871+00:00", "service": "aggregator", "correlation_id": "demo-soutenance", "event": "http_request", "path": "/api/v1/dashboard/structures/1", "status": 200, "duration_ms": 43.2}
indicateurs-api | {"timestamp": "2026-10-21T13:09:15.847+00:00", "service": "indicateurs-api", "correlation_id": "demo-soutenance", "event": "http_request", "path": "/api/v1/indicateurs/structures/1", "status": 200, "duration_ms": 1.3}
structures-api | {"timestamp": "2026-10-21T13:09:15.856+00:00", "service": "structures-api", "correlation_id": "demo-soutenance", "event": "http_request", "path": "/api/v1/structures/1", "status": 200, "duration_ms": 1.1}

Les lignes sont raccourcies (« … » et champs secondaires retirés) ; la ligne de Kong compte à l'origine plus de 2 000 caractères. L'identifiant `demo-soutenance` apparaît dans les cinq composants : Kong, l'agrégateur et les trois API.

B.8 Observation : les sondes de santé dans les journaux

Pendant la démonstration, le terminal 1 a affiché environ 870 lignes, dont 810 correspondaient aux appels `GET /health` que Docker envoie toutes les 10 secondes pour vérifier l'état des conteneurs. Ces lignes noyaient les journaux utiles. Le fichier `common/observability.py` a donc été modifié pour ne plus journaliser `/health` ; la sonde continue de fonctionner, seule sa trace disparaît. Les 23 tests unitaires passent toujours après cette modification.

Les 7 scénarios ont été rejoués le même jour après reconstruction des images (`docker compose up -d --build`), avec des résultats identiques. Le terminal 1 n'a plus affiché que 55 lignes : exactement 5 par appel au tableau de bord (trois `upstream_call`, un `aggregation_completed`, un `http_request`) pour 11 appels, et aucune ligne `/health`.

Au scénario 7, la recherche de `demo-soutenance` a alors renvoyé deux lignes de Kong. La première date de la démonstration précédente (13 h 09) : le conteneur Kong n'a pas été recréé et a gardé ses anciens journaux. Pour la soutenance, il suffit d'utiliser un identifiant jamais employé, par exemple `demo-jury`.